

Unit-II Exception Handling, Thread, I/O

Introduction to Exception:

While doing programming in Java, there is possibility to happen errors. These errors may be encountered at two times-

- 1) Compile time error
- 2) Runtime error

Let's see these types of errors in details-

1) Compile time error:

- These types of errors detected by compiler at compile time of program.
- Such type of errors is due to wrong syntax of programming languages.
- Or when programmer violate the rules of writing Java statements.
- These compiler error indicates something that must be fixed before the code can be compiled.
- All these errors are detected by compiler and thus are known as "compile-time errors"
- Most frequent compile time errors happen if we make following mistake:
 - Missing Parenthesis {}
 - Use of variable without declaring it.
 - Missing semicolon at end of statement.
- Note that, such type of errors are handled by compiler and it generate corresponding error message such that we can correct it.
- These types of errors are not harmful for computer because it is easily detected by compiler at compile time.

2) Run time error (Exception):

- Exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run time and that disturbs the normal flow of the program's instructions.
- An exception is a problem or error that arises during the execution of a program (program running).
- **Exceptions** are run-time abnormal conditions that a program encounters during its execution.
- When an Exception occurs the normal flow of the program is disturbed and the program/Application/software terminates abnormally that cause's application failure which is harmful, which is not recommended, therefore, these exceptions should to be handled.
- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
 - A user has entered an invalid data.
 - A number is divided by zero.
 - Attempt to retrieve array element from invalid index.
 - Attempt to open a file which is not available.
 - A network connection has been lost in the middle of communications.

What happen if we do not handle exceptions?

- When an exception occurred, if we don't handle it, the program terminates abnormally and remaining program code will not get executed that cause's application failure which is harmful.

Example-

- We know that, an array is of fixed size and each element is accessed using the indices. Consider, we have created an array with size 7. Then the valid expressions to access the elements of this array will be a[0], a[1], .. to a[6] (i.e. length-1).
- Whenever, we used a negative index value or, the index value greater than the size of the array, then there will be exception occurs.

Types of Exceptions:

Java has two types of exceptions.

1. Checked exception
2. Unchecked exception

Let's see these exceptions in details-

1) Checked Exceptions:

- The exception which occurs at compile time of program is called "Checked Exception".
- **Java compiler** checks whether a program contains the checked exception or not at the time of compilation.
- Developer has overall control on checked exception because these occur at compile time.
- For example: SQLException, IOException, ClassNotFoundException, FileNotFoundException, MalformedURLException etc. these are the types of Checked exception.

Example: Sample program for checked exception.

```
import java.io.*;
public class CheckExpDemo
{
    public static void main(String args[])
    {
        FileInputStream in = new FileInputStream("read.txt");
        int c;
        while((c=in.read())!=-1)
        {
            System.out.println((char)c);
        }
        in.close();
    }
}
```

Above program generates compile time exception like-
error: unreported exception IOException
 Because **IOException** must be **caught or declared to be thrown**.
 Here, IOException occur at compile time therefore it is checked exception.

2. Unchecked Exceptions:

- The exception which occurs at run time of program is called "Unchecked Exception".
 - JVM finds unchecked exception at runtime of program.
 - Such exception occurs due to logical errors or improper use of API etc.
 - Developer has no control on unchecked exception because these occur at run time of program.
- For example: ArrayIndexOutOfBoundsException, NullPointerException, ArithmeticException.
 Example : Sample program for unchecked exception

```
class UnCheckExpDemo
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 0;
        int c = a/b;
        System.out.println(c);
    }
}
```

Above program raise run time exception like-
Exception in thread "main" java.lang.ArithmeticException: / by zero

Difference between error and exception:

Error	Exception
It cannot be caught.	It can be handled by try and catch block.
Errors are by default unchecked type .	Exception can be either checked or unchecked type.
It is defined in java.lang.Error package.	It is defined in java.lang.Exception package.
Errors are generally caused by environment (Operating system) in which application is running.	Exceptions are generally caused by application itself .
Example: StackOverflowError, OutOfMemoryError etc.	Example: SQLException, IOException, ClassCastException, ArrayOutOfBoundsException

Exception Handling in Java:

- Exception handling is the mechanism to handle the abnormal termination of the program without failure.
- By handling exception, we can protect our application from failure.
- In Java, there are different kinds of exceptions which are handled by **Exception** class which was derived from **Throwable** class.
- The Exception class Hierarchy in Java is shown as-

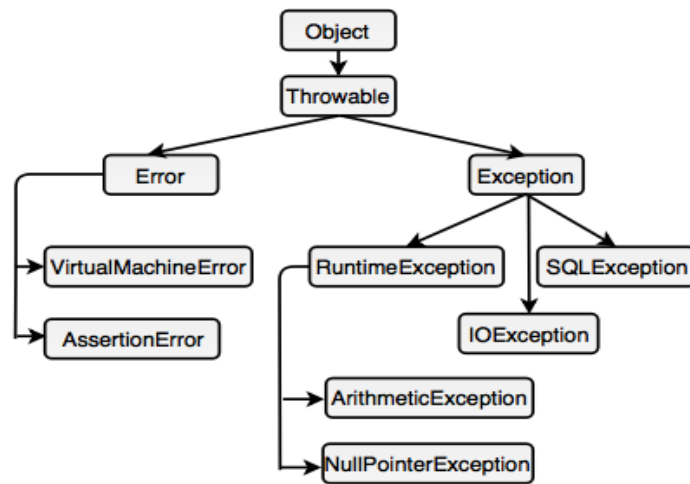


Fig: Exception Hierarchy

- To protect application from failure, it is necessary to handle the exception.
- In java, exceptions are handled using five keywords which are as follow-
 - 1) **try**
 - 2) **catch**
 - 3) **throw**
 - 4) **throws**
 - 5) **finally**

Let's see these keywords in details-

1) The **try** block:

- The try block contains the code or statements there might be possibility to occur an exception.
- The try block must have least one catch block or finally block.

2) The **catch** block:

- A catch block must be defined after try block. It contains the error handling code.
- A catch block executes if and only if an exception occurs in corresponding try block.
- Syntax of try and catch:

```

try
{
    //code that cause exception;
}
catch(Exception_type e)
{
    //exception handling code
}
  
```

Built-in exceptions:

- Built-in exceptions are the exceptions which are available in Java libraries. These exceptions are suitable to explain certain error situations.
- ***In Java, all exceptions are handled by exception built-in classes and these are subclasses of *Exception* class.***
- Below is the list of important built-in exceptions in Java.

1. **ArithmeticException**

It is thrown when an exceptional condition has occurred in an arithmetic operation.

2. **ArrayIndexOutOfBoundsException**

It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

3. **ClassNotFoundException**

This Exception is raised when we try to access a class whose definition is not found.

4. **FileNotFoundException**

This Exception is raised when a file is not accessible or does not open.

5. **IOException**

It is thrown when an input-output operation failed or interrupted.

6. **InterruptedException**

It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.

7. NoSuchElementException

It is thrown when a class does not contain the field (or variable) specified.

8. NoSuchMethodException

It is thrown when accessing a method which is not found.

9. NullPointerException

This exception is raised when referring to the members of a null object. Null represents nothing.

10. NumberFormatException

This exception is raised when a method could not convert a string into a numeric format.

11. RuntimeException

This represents any exception which occurs during runtime.

12. StringIndexOutOfBoundsException

It is thrown by String class methods to indicate that an index is either negative than the size of the string

Examples of Built-in Exception:

<pre>// Java program to demonstrate ArithmeticException class ArithmeticException_Demo { public static void main(String args[]) { try { int a = 30, b = 0; int c = a/b; // cannot divide by zero System.out.println ("Result = " + c); } catch(ArithmeticException e) { System.out.println ("Can't divide a number by 0"); } } }</pre> Output: Can't divide a number by 0	<pre>//Java program to demonstrate NullPointerException class NullPointer_Demo { public static void main(String args[]) { try { String a = null; //null value System.out.println(a.charAt(0)); } catch(NullPointerException e) { System.out.println("NullPointerException !!"); } } }</pre> Output: NullPointerException !!
<pre>//Java program to demonstrate StringIndexOutOfBoundsException class StringIndexOutOfBoundsDemo { public static void main(String args[]) { try { String a = "Box is Heavy"; // length is 12 char c = a.charAt(20); // 20th index element System.out.println(c); } catch(StringIndexOutOfBoundsException e) { System.out.println("StringIndexOutOfBoundsException"); } } }</pre> Output: StringIndexOutOfBoundsException	<pre>//Java program to demonstrate NumberFormatException class NumberFormat_Demo { public static void main(String args[]) { try { int n= Integer.parseInt ("GOOD"); System.out.println(n); } catch(NumberFormatException e) { System.out.println("Number format exception.."); } } }</pre> Output: Number format exception..
<pre>// Java program to demonstrate ArrayIndexOutOfBoundsException class ArrayIndexOutOfBounds_Demo { public static void main(String args[]) { try { int a[] = new int[5]; a[6] = 9; // assigning 9 at 6th index but array size is 5 } } }</pre>	<pre>// Java program to demonstrate FileNotFoundException import java.io.*; class FileNotFoundException_Demo { public static void main(String args[]) { try { FileReader in=new FileReader("Read.txt"); } } }</pre>

<pre> } catch(ArrayIndexOutOfBoundsException e) { System.out.println ("Array Index is Out Of Bounds"); } } } } Output: Array Index is Out Of Bounds </pre>	<pre> } catch(FileNotFoundException e) { System.out.println("File Not found.."); } } } } Output: File Not found.. </pre>
---	---

Multiple catch blocks:

- In a single program, there is a possibility to occur multiple exceptions of different types. Then to handle such multiple exceptions, we require multiple catch blocks.
- A single try block has multiple catch blocks to handle different types of exceptions.
- Syntax:

```

try
{
    // code which generate exception
}
catch(Exception_type1 e1)
{
    //exception handling code
}
catch(Exception_type2 e2)
{
    //exception handling code
}

```

Example: Following program illustrating multiple catch blocks that handles different exception.

```

import java.util.*;
public class MultiCat
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        int a,b,c;

        try
        {
            System.out.println("Enter Two NO=");
            a = sc.nextInt();
            b = sc.nextInt();
            c = a / b;
            System.out.println("Result = "+c);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Number cannot divide by zero.");
        }
        catch(InputMismatchException e) //Note: InputMismatchException is found under java.util package
        {
            System.out.println("Enter only numeric value.");
        }
    }
}

```

Output:

Enter Two NO= 6 2 Result = 3	Enter Two NO= 7 0 Number cannot divide by zero.	Enter Two NO= 5 k Enter only numeric value.
------------------------------------	---	---

3) The **finally** block:

- The code present in finally block will always be executed even if try block has exception or not.
- Basically, finally block contains important code like closing file, closing database connection, taking back-up of database etc.
- finally block executes after try and catch block and hence, finally block must be followed by try or catch block.
- Syntax:

```
try
{
    // code
}
catch(Exception_type1 ex)
{
    // catch block1
}
catch(Exception_type2 ex)
{
    //catch block 2
}
finally
{
    //finally block
    //always execute
}
```

Example: A program that demonstrate use of finally block

```
class FinallyTest
{
    public static void main(String args[])
    {
        int arr[] = new int[5];
        try
        {
            arr[7]=10;
        }
        catch(ArrayIndexOutOfBoundsException ai)
        {
            System.out.println("Exception thrown : " + ai);
        }
        finally
        {
            System.out.println("The finally block is executed...");
        }
    }
}
```

Output:

Exception thrown : java.lang.ArrayIndexOutOfBoundsException: 7
The finally block is executed...

4) 'throw' keyword in Java:

- The **throw** keyword in Java is used to explicitly or forcefully throw the exception.
- That is with the help of **throw** any kind of exception like `ArithmeticException`, `FileNotFoundException` etc. can be explicitly thrown.
- It can be used for both checked and unchecked exception.

Syntax:

throw new Exception_subclass();

Here,

`throw` is keyword that throws exception.
`new` is operator
`Exception_subclass` is any exception class.

Example: A program to illustrate throw keyword in java

```
public class throwDemo
{
    public static void main(String args[])
    {
        try
        {
            throw new ArithmeticException("Not valid.. ");
        }
        catch(ArithmeticException e)
        {
            System.out.println("ArithmeticException occur "+e);
        }
    }
}
```

OUTPUT:

ArithmeticException occur java.lang.ArithmeticException: Not valid..

5) 'throws' keyword in Java

- The **throws** keyword is **generally used for handling checked exception**.
- Without handling checked exceptions program can never be compiled.
- If we do not want to handle exception by *try* and *catch* block, then it can be handled by *throws*.
- The **throws** keyword is always added after the method signature and it **tells the caller method that the method might throw certain exceptions hence it is used after caller method also.**
- Syntax of throws:

```
Return_type    method_name() throws Exception_Class_Name
{
    Body of method
}
```

Example: A program to illustrate uses of throws keyword

```
import java.io.*;
class MyClass
{
    void method() throws IOException
    {
        System.out.println("My method...");
    }
}
class throwsDemo
{
    public static void main(String args[]) throws IOException
    {
        MyClass m=new MyClass();
        m.method();
        System.out.println("normal flow...");
    }
}
```

Difference between throw and throws keyword in Java:

Throw	Throws
It is used to throw own exception.	It is used when program does not handle exception via try block.
It explicitly throw the exceptions.	It handles all exceptions generated by program.
It cannot throw multiple exceptions at a time.	It Throws multiple exception E.g. public void method() throws IOException, SQLException
It is used within the method.	It is used at method signature.

Creating Custom Exceptions: (User defined exception)

- The exceptions that are created or defined by the programmer is known as “Custom exception.”
- To create user defined exception, “Custom exception class” must be extended from “Exception” class.
- These exceptions are created to generate a solution as per programmer’s need.
- Syntax to create custom exception:

```
class Custom_Except_ClassName extends Exception
{
    Body of exception class
}
```

Example 1: Program to create custom exception in Java

```
import java.util.Scanner;
class custom extends Exception
{
    public custom(String msg)
    {
        super(msg);
    }
}
public class MyExcept
{
    public static void main(String arg[])
    {
        Scanner sc=new Scanner(System.in);
        int age;
        try
        {
            System.out.println("Enter your Age");
            age=sc.nextInt();
            if(age<18)
            {
                throw new custom("Age should be greater than 18..");
            }
            else
            {
                System.out.println("You are Eligible for voting..");
            }
        }
        catch(custom e)
        {
            System.out.println(e);
        }
    }
}
```

OutPut:

Enter your Age 52 You are Eligible for voting..	Enter your Age 15 custom: Age should be greater than 18..
---	---

Example-2: Write a Java program that takes a string as input and throws an exception if the string *does not contain vowels*.

```
import java.util.Scanner;
class StrExcept extends Exception
{
    StrExcept(String msg)
    {
        super(msg);
    }
}
```



```

class StrVol
{
    public static void main(String ar[])
    {
        Scanner sc=new Scanner(System.in);
        String str;
        int temp=0;
        try
        {
            System.out.println("Enter any string..");
            str=sc.next();
            int len=str.length();
            for(int i=0;i<=len-1;i++)
            {
                if(str.charAt(i)=='a'||str.charAt(i)=='e'||str.charAt(i)=='i'||str.charAt(i)=='o'||str.charAt(i)=='u')
                {
                    temp=1;
                    break;
                }
            }
            if(temp==0)
            {
                throw new StrExcep("String does not contain vowels..");
            }
            else
            {
                System.out.println("String has Vowels...");
            }
        }
        catch(StrExcep e)
        {
            System.out.println(e);
        }
    }
}

```

OutPut:

Enter any string.. xyz StrExcep: String does not contain vowels..	Enter any string.. sangola String has Vowels...
--	--

Thread:

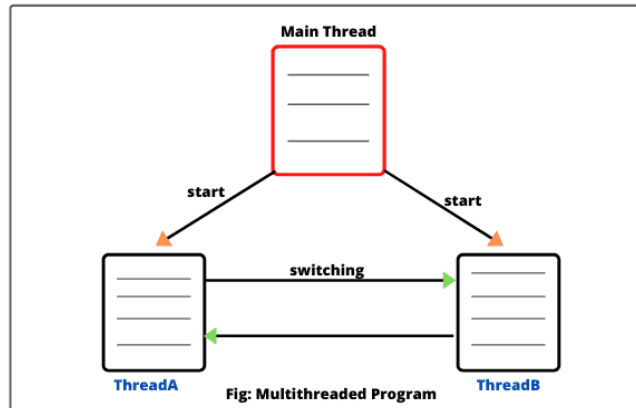
- We know that, to get result or output from method, its statements must be executed. And **Thread is such a concept which is responsible to execute the statements** of method.
- In short, we can say that because of thread methods of program are executes and we will get result.
- C, C++ language supports for single threading concept i.e. to execute entire C, C++ program there is only one thread that execute entire statements of program.
- Java language supports for **multithreading** concept i.e. to execute Java program there will be allocate more threads that can execute multiple methods of program concurrently. (All methods will execute parallel with the help of threads)
- **Definition: Thread means a smallest unit of execution inside a program and it allows a program to do multiple tasks at the same time.**

Example-

- In Java, the default thread that executes your java program is called the **main thread**.
- When you run a Java program, the Java Virtual Machine (JVM) automatically creates a thread called the **main thread**. This thread starts executing the program from the main() method.

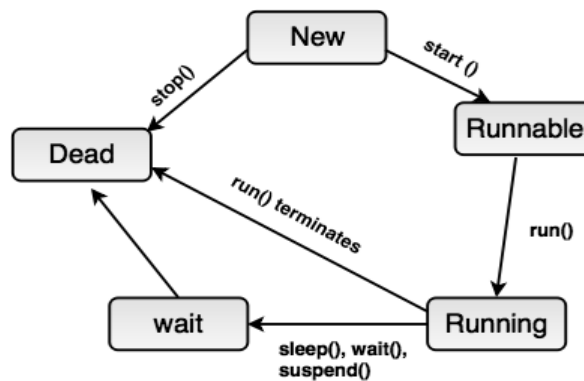
Multithreading:

- Multithreading means executing multiple parts (methods) of the same program concurrently with the help of threads.
- Multithreading is a part of multitasking i.e. at one time multiple tasks were carried out concurrently.
- Single java program may contain more than one thread and each thread complete its task simultaneously.
- Following diagram shows multithreading:



Life Cycle of Thread:

- Life Cycle of Thread in Java represents all states of a thread from birth to its death.
- Every thread in Java has life cycle which is shown in following diagram:



Thread life cycle is explained bellow-

- 1) **New State:** When we create a thread, then it is said to be in the new state. The **start()** method creates thread.
- 2) **Runnable State:** The **start()** method invokes the **run()** method, then created thread now in runnable state.
- 3) **Running State:** When the thread is currently running its task, then it is in running state.
- 4) **Wait State:** When other threads are holding the resources, the current thread is said to be in wait state
- 5) **Dead State:** When the thread has completed its task, then it is said to be in dead state.

Thread Methods in Java:

For multithreading, java provides built-in **Thread** class that have some important methods as follow-.

Methods	Description
public void start()	The start() method creates thread. This method invokes the run() method, which causes the thread to begin its execution.
public void run()	Contains the actual functionality of a thread.
public static void sleep (long millisec)	<u>Blocks currently running thread for at least certain millisecond.</u>
public static void yield ()	Causes the <u>current running thread to halt (pause) temporarily</u> and allow other threads to execute.
public static Thread currentThread ()	It <u>returns the reference of the current running thread</u> object.
public final void setPriority (int priority)	<u>Changes the priority of the current thread.</u> Note: The minimum priority number is 1 and the maximum priority number is 10 .
public final boolean isAlive ()	Used <u>to know whether a thread is live or not.</u>

	It <u>returns true if the thread is alive</u> . A thread is said to be alive if it has been started but has not yet died.
public void interrupt ()	<u>Interrupts the thread, causing it to continue its execution</u> if it was blocked for some reason.

Creating Thread:

There are two ways to create a thread in Java.

1. By **extending** a class from **Thread class**.
2. By **implementing** a class from **Runnable interface**.

Let's see this way in details-

1. Extending a class from Thread class:

We can create a new thread by **extending from Thread class**.

- Thread class has constructor and method to create and perform the operation on thread.
- Following program shows creating Thread by extending Thread class

<pre> class MyThread extends Thread { public void run() { for(int i=1;i<=20;i=i+5) { System.out.print(" "+i); } } } </pre>	<pre> public class ThreadDemo { public static void main(String[] args) { MyThread t1 = new MyThread(); MyThread t2 = new MyThread(); MyThread t3 = new MyThread(); t1.start(); //invokes run() t2.start(); //invokes run() t3.start(); //invokes run() } } </pre> OUTPUT: Whenever we run this program, we will get different-different outputs. Since, Thread t1 and t2 parallelly execute same block of code.
--	---

2. Implementing a class from Runnable Interface:

- We can also create a new thread by **implementing Runnable interface**.
- Runnable interface is available in *java.lang* package.
- The purpose of Runnable interface is to provide a set of rules common for objects to execute the functionality of thread while they are active.
- Example : Sample program to create thread class by implementing Runnable interface

<pre> class MyThread implements Runnable { public void run() { for(int i=1;i<=20;i=i+5) { System.out.print(" "+i); } } } </pre>	<pre> public class MyRunThread { public static void main(String[] args) { MyThread dt1=new MyThread(); MyThread dt2=new MyThread(); Thread t1 = new Thread(dt1); Thread t2 = new Thread(dt2); t1.start(); t2.start(); } } </pre> OUTPUT: Whenever we run this program, we will get different-different outputs. Since, Thread t1 and t2 parallelly execute same block of code.
---	--

In above program:

- Created **Runnable tasks** (dt1, dt2)
- Pass them to **Thread objects** (t1, t2)

Note: Thread class is responsible for **starting a new thread** therefore it takes a Runnable object (dt1,dt2) as input.

Thread Priorities:

- Thread priorities are the **integer number from 1 to 10 assigned** to thread which helps **to determine the order in which threads are to be scheduled for running**.
- **Default priority of a thread is 5 (NORM_PRIORITY)**. The value of **MIN_PRIORITY is 1** and the value of **MAX_PRIORITY is 10**.
- In most cases, the thread scheduler, schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.
- Thread scheduler selects the thread for execution on the first-come, first-serve basis. That is, the threads having equal priorities share the processor time on the first-come, first-serve basis.
- When multiple threads are ready for execution, the highest priority thread is selected and executed by JVM. In case when a high priority thread stops, yields, or enters into the blocked state, a low priority thread starts executing.
- If any high priority thread enters into the runnable state, it will preempt (prevent) the currently running thread forcing it to move to the runnable state. Note that the highest priority thread always preempts any lower priority thread.

Setter & Getter Method of Thread Priority:

- The Thread class has getter and setter methods related with Thread priority which are discussed below-

1) int getPriority():

- This is method of Thread class which returns the priority of the given thread.

2) void setPriority(int newPriority):

- This is also method of Thread class that updates or assign the newPriority for the thread.
- The method throws `IllegalArgumentException`, if the value `newPriority` goes out of the range (i.e other than 1 to 10).
- Following program shows use of `MAX_PRIORITY`, `MIN_PRIORITY` and `NORM_PRIORITY`

```
class ThreadPriorityDemo extends Thread
{
    public void run()
    {
        System.out.println(Thread.currentThread()+" Priority "+Thread.currentThread().getPriority());
    }
}
Class myThread
{
    public static void main(String args[])
    {
        ThreadPriorityDemo t1=new ThreadPriorityDemo ();
        ThreadPriorityDemo t2=new ThreadPriorityDemo ();
        ThreadPriorityDemo t3=new ThreadPriorityDemo ();
        t1.setPriority(Thread.MAX_PRIORITY);
        t2.setPriority(Thread.MIN_PRIORITY);
        t3.setPriority(Thread.NORM_PRIORITY);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT: (Note: output is shown in form Thread[thread-name, priority, thread-group])

Thread[Thread-0,10,main] Priority 10

Thread[Thread-2,1,main] Priority 1

Thread[Thread-1,5,main] Priority 5

Example : Following program shows use of `getPriority()` and `setPriority()` method.

```
class newThread extends Thread
{
    public void run()
    {
        System.out.println(Thread.currentThread()+" has Priority "+Thread.currentThread().getPriority());
    }
}
```

```

public class AllThread
{
    public static void main(String []ar)
    {
        newThread t1=new newThread();
        newThread t2=new newThread();
        newThread t3=new newThread();
        newThread t4=new newThread();
        t1.setPriority(Thread.MAX_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MIN_PRIORITY);
        t4.setPriority(4);
        t1.start();
        t2.start();
        t3.start();
        t4.start();
    }
}

```

OUTPUT:

Thread[Thread-0,10,main] has Priority 10
 Thread[Thread-1,5,main] has Priority 5
 Thread[Thread-3,4,main] has Priority 4
 Thread[Thread-2,1,main] has Priority 1

Example : Following program shows execution of thread depending on their priority.

```

public class ThreadExec extends Thread
{
    public void run()
    {
        System.out.println(Thread.currentThread());
    }
    public static void main(String[] args)
    {
        ThreadExec t1 = new ThreadExec();
        ThreadExec t2 = new ThreadExec();
        ThreadExec t3 = new ThreadExec();

        t1.setPriority(4);
        t2.setPriority(2);
        t3.setPriority(8);

        t1.start();
        t2.start();
        t3.start();
    }
}

```

Note that: By executing above program again and again, we will get different-different results.

OUTPUT:

Thread[Thread-2,8,main]	Thread[Thread-0,4,main]	Thread[Thread-2,8,main]
Thread[Thread-0,4,main]	Thread[Thread-2,8,main]	Thread[Thread-1,2,main]
Thread[Thread-1,2,main]	Thread[Thread-1,2,main]	Thread[Thread-0,4,main]

Thread Synchronization: (Lock-Unlock object)

- We know that in multi-threaded program, multiple threads try to access the same resources (like variable, memory, printer etc.) at same time and finally produce incorrect and unpredicted results.
- Therefore, it is necessary to allow only one thread can access the resource at one time where as other threads may remain in waiting state. And this task in Java is done with the help of Synchronization. And it is achieved by synchronized block and synchronized method.
- Definition:** Thread synchronization is such a mechanism that allows only one thread to act on object or shared resources while other threads are in waiting state. The next thread only allows to enter in shared resources, if and only if current working thread finishes its task.
- In short, because of Thread synchronization multiple threads cannot enters in shared resources at one time.

- Synchronized blocks in Java are marked with the **synchronized** keyword.
- All synchronized blocks synchronize on the same object can allow only have one thread executing inside them at a time. All other threads attempting to enter the synchronized block are blocked until the thread inside the synchronized block exits that block.
- Thread synchronization achieved by two ways-
1) synchronized block 2) synchronized method.
- Let's see these in details-

1) Thread synchronization by synchronized block-

Following is the general form of a synchronized block:

```
synchronized(sync_object)
{
    // Access shared variables and other shared resources
}
```

From above syntax-

- Only one thread can execute synchronized block at a time.
- *Sync_object* is a reference to an object of class whose object should be locked.
- This **synchronization is implemented in Java with a concept called monitors**. Only one thread can own a monitor at a given time.
- **When a thread acquires a lock, it is said to have entered the monitor**. All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor.
- Following is an example of **multithreading with synchronized block**:

```
class ThreadJob
{
    public void myjob()
    {
        for(int i=11;i<=15;i++)
        {
            System.out.println(i);
        }
    }
}

class newThread extends Thread
{
    ThreadJob x;
    newThread(ThreadJob y)
    {
        x=y;
    }
    public void run()
    {
        synchronized(x)
        {
            System.out.println(Thread.currentThread());
            x.myjob();
        }
    }
}

public class ThreadSyncDemo
{
    public static void main(String []arg)
    {
        ThreadJob obj=new ThreadJob();
        newThread t1=new newThread(obj);
        newThread t2=new newThread(obj);
        newThread t3=new newThread(obj);
        newThread t4=new newThread(obj);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

```

        t4.start();
    }
}
OUTPUT:
Thread[Thread-0,5,main]
11
12
13
14
15
Thread[Thread-2,5,main]
11
12
13
14
15
Thread[Thread-1,5,main]
11
12
13
14
15
Thread[Thread-3,5,main]
11
12
13
14
15

```

2) Thread synchronization by Synchronization Method:

- This is another way of doing multithreading with synchronized method.
- If we declare any method using **synchronized** keyword then it is known as “synchronized method”
- The working of synchronized block and Synchronized method is same which is used to lock an object for any shared resource if any thread enters it.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

```

class ThreadDemo2 extends Thread
{
    synchronized void threadWork()
    {
        System.out.println("Active Thread="+Thread.currentThread());
        for(int i=11;i<=15;i++)
        {
            System.out.println(i);
        }
    }
    public void run()
    {
        threadWork();
    }
}

public class MultiThread
{
    public static void main(String arg[] )
    {
        ThreadDemo2 t1=new ThreadDemo2();
        ThreadDemo2 t2=new ThreadDemo2();
        t1.start();
        t2.start();
    }
}

```

```
}  
}
```

OUTPUT:

Active Thread=Thread[Thread-0,5,main]

11

12

13

14

15

Active Thread=Thread[Thread-1,5,main]

11

12

13

14

15

Thread Communication: (Inter-Thread Communication)

- Inter-thread communication in Java is a technique in which multiple threads communicate with each other to achieve specific task.
- It provides an efficient way to communicate threads with each other by reducing CPU idle time. (CPU idle time is a process in which CPU cycles are not wasted.)
- When more than one threads are executing simultaneously, sometimes they need to communicate with each other by exchanging information with each other. A thread exchanges information before or after it changes its state.
- **For example**, suppose, there are two threads A and B. Thread B uses data produced by Thread A and performs its task.
- If Thread B waits for Thread A to produce data, it will waste many CPU cycles. But if threads A and B communicate with each other when they have completed their tasks, they do not have to wait and check each other's status every time.
- Thus, CPU cycles will not waste. This type of information exchanging between threads is called "inter-thread communication" in Java
- Inter thread communication is used when an application has two or more threads that exchange same information.
- Inter thread communication helps in avoiding thread pooling.
- Inter thread communication in Java can be achieved by using three methods provided by Object class of java.lang package. They are:

1. **wait()**

2. **notify()**

3. **notifyAll()**

- These methods can be called only within a synchronized method or synchronized block of code otherwise, an exception named *IllegalMonitorStateException* is thrown.
- All these methods are declared as final. Since, it throws a checked exception.
- Let's see these methods in details-

1. **wait() Method:**

- wait() method in Java keeps current thread in waiting state.
- This method will throws *InterruptedException*.
- Various overloaded forms of wait() method are as follows:

public final void wait()

public final void wait(long millisecond) throws InterruptedException

public final void wait(long millisecond, long nanosecond) throws InterruptedException

- All overloaded forms of wait() method throws *InterruptedException*.
- If time is not specified in the wait() method, a thread can wait for maximum time.

2. **notify() Method:**

- The notify() method wakes up a single thread which is already in wait() state. If more than one thread is waiting, this method will awake one of them.
- The prototype of notify() method is as follows:

public final void notify();

3. notifyAll() Method:

- The notifyAll() method is used to wake up all threads which are already in wait() state.
- The prototype of notifyAll() method is as follows:

public final void notifyAll()

Following program shows inter thread communication between producer and consumer thread

```
class producer extends Thread
{
    StringBuffer sb; //StringBuffer is 'Mutable' i.e. value stored in its object can be changed
    producer()
    {
        sb=new StringBuffer(); //allocate memory
    }
    public void run()
    {
        synchronized(sb)
        {
            try
            {
                for(int i=1;i<=10;i++)
                {
                    Thread.sleep(2000); //it will cause InterruptedException
                    sb.append(i+":");
                    System.out.println("Producing.." +i);
                }
                System.out.println("Production is over...");
                sb.notify(); //production is Over & now notify to consumer thread
            }
            catch (InterruptedException e)
            {
                System.out.println(e);
            }
        }
    }
}

class consumer extends Thread
{
    producer p;
    consumer(producer x)
    {
        p=x;
    }
    public void run()
    {
        synchronized(p)
        {
            try
            {
                p.wait(); // Wait till notification is received
                System.out.println("Consumer received product: " +p.sb);
            }
            catch (InterruptedException e)
            {
                System.out.println(e);
            }
        }
    }
}

class ThreadComm
{
}
```

```

public static void main(String ar[])
{
    producer t1=new producer();
    consumer t2=new consumer(t1);
    t1.start();
    t2.start();
}
}

```

OUTPUT:

```

Producing..1
Producing..2
Producing..3
Producing..4
Producing..5
Producing..6
Producing..7
Producing..8
Producing..9
Producing..10
Production is over...
Consumer received product: 1:2:3:4:5:6:7:8:9:10:

```

Input/Output (I/O) in Java:

Input / Output Handling

- We know to solve any problem; we need an input and output operations. To perform input and output (I/O) in Java, it has *java.io* package that contains the classes which performs I/O.
- Such I/O operations done via streams (Sequence of byte) that represents an input source and an output destination. The stream in the java.io package supports many data such as primitives, object, localized characters, etc.
- Java Program performs input / output through stream. So, let's understand stream.

Stream:

- A stream can be defined as a sequence of bytes (data) which flows from input device to output device and vice-versa.
- There are two kinds of Streams:
- **InPutStream:** The InputStream is used to read data from a source i.e. from keyboard, file etc.
- **OutPutStream:** The OutputStream is used for writing data to a destination i.e. to monitor, file etc.
- Following diagram shows I/O stream in Java-



- To perform both Input stream and Output stream operations, Java language has two classes viz.-
 - 1) Byte Stream (Binary Stream) class
 - 2) Character Stream (Text stream) class
- Before learning these classes, we have to know file and its type which is discussed below-

File:

- File is a place on secondary storage (Hard disk, CD, DVD, Pendrive etc.) where large amount of data can be stored permanently.

Types of file:

There are two types of files-

- 1) Binary files

2) Text Files

Let's see these types in details:

1) Binary Files:

- The files which operate the data in terms of binary stream (8 bit data) such files are called "Binary Files"
- Binary files used to operate binary data such as images, videos etc.
- Binary files are secured file because data in not directly interpreted by human being.

2) Text files:

- The files which operate the data in terms of text streams (16 bit Unicode data) or characters such files are called "Text files".
- Text files are not secured file because its data directly interpreted by human being.

To deal with file in java, it has following classes:

1) Byte Streams class:

- Java byte streams are used to perform input and output in terms of byte (8-bits).
- Though there are many classes related to byte streams but the most frequently used classes are, *FileInputStream* and *FileOutputStream*.
- Note: The file I/O operation throws the *IOException*
- Following program shows use of *ByteStream* classes.

Program 1) Program that reads content of file.

```
import java.io.*;
public class FileRead
{
    public static void main(String args[]) throws IOException
    {
        int c;
        try
        {
            FileInputStream in = new FileInputStream("d:\\myfile\\input.txt");
            while ((c = in.read()) != -1) //While not end of File
            {
                System.out.print((char)c);
            }
            in.close();
        }
        catch(FileNotFoundException ex)
        {
            System.out.println(ex);
        }
    }
}
```

Program 2) Program that write content into file.

```
import java.io.*;
public class FileWrite
{
    public static void main(String args[]) throws IOException
    {
        int i;
        try
        {
            FileOutputStream out = new FileOutputStream("d:\\myfile\\write.txt");
            String str="Box is Heavy";
            int len=str.length();
            for(i=0;i<=len-1;i++)
            {
                out.write(str.charAt(i));
            }
            out.close();
        }
        catch(FileNotFoundException ex)
        {
            System.out.println(ex);
        }
    }
}
```

```

    {
        System.out.println(ex);
    }
}
}

```

Program 3) Program that copies content of file into another file.

```

import java.io.*;
public class FileCopy
{
    public static void main(String args[]) throws IOException
    {
        int c;
        try
        {
            FileInputStream in = new FileInputStream("d:\\myfile\\input.txt");
            FileOutputStream out = new FileOutputStream("d:\\myfile\\output.txt");

            while ((c = in.read()) != -1)    //while there is not end of File
            {
                out.write(c);
            }

            in.close();
            out.close();

        }
        catch(FileNotFoundException ex)
        {
            System.out.println(ex);
        }
    }
}

```

2) Character Streams class:

- We know that, Java Byte streams are used to perform input and output in terms of bytes (8-bit), whereas Java Character streams are used to perform input and output in terms 16-bit Unicode text.
- There are many classes related to character streams but the most frequently used classes are, *FileReader* and *FileWriter*.
- **Note:** internally *FileReader* uses *FileInputStream* and *FileWriter* uses *FileOutputStream* but there is **major difference is that *FileReader reads two bytes at a time* and *FileWriter writes two bytes at a time***.
- Following programs shows use of *CharacterStream* classes:

Program 1) Program that read the content of file.

```

import java.io.*;
public class NewReadFile
{
    public static void main(String args[]) throws IOException
    {
        int c;
        try
        {
            FileReader in = new FileReader("d:\\myfile\\read.txt");
            while ((c = in.read()) != -1)    //while not end of file
            {
                System.out.print((char)c);
            }
            in.close();
        }
        catch(FileNotFoundException ex)
        {
            System.out.println(ex);
        }
    }
}

```

```
}  
}  
}
```

Program 2) Program to write the content in file.

```
import java.io.*;  
public class NewWriteFile  
{  
    public static void main(String args[]) throws IOException  
    {  
        int i;  
        try  
        {  
            FileWriter out = new FileWriter("d:\\myfile\\NewWrite.txt");  
            String str="Box is Heavy";  
            int len=str.length();  
            for(i=0;i<=len-1;i++)  
            {  
                out.write(str.charAt(i));  
            }  
            out.close();  
        }  
        catch(FileNotFoundException ex)  
        {  
            System.out.println(ex);  
        }  
    }  
}
```

Program 3) Program that copies content of file into another file.

```
import java.io.*;  
public class NewCopyFile  
{  
    public static void main(String args[]) throws IOException  
    {  
        int c;  
        try  
        {  
            FileReader in = new FileReader("d:\\myfile\\read.txt");  
            FileWriter out = new FileWriter("d:\\myfile\\write.txt");  
            while ((c = in.read()) != -1) //while not end of file  
            {  
                out.write(c);  
            }  
            in.close();  
            out.close();  
        }  
        catch(FileNotFoundException ex)  
        {  
            System.out.println(ex);  
        }  
    }  
}
```

// Following Java program reads a file and **throws an exception if the file is empty.**

```
import java.io.*;
class myExp extends Exception
{
    myExp(String msg)
    {
        super(msg);
    }
}
class File4
{
    public static void main(String ar[]) throws IOException
    {
        try
        {
            FileReader in=new FileReader("d:\\filedata\\checkfile.txt");
            int ch;
            ch=in.read();
            if(ch==-1)
            {
                throw new myExp("Your File Is Empty...");
            }
            else
            {
                System.out.println("File has data..");
            }
            in.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println(e);
        }
        catch(myExp e)
        {
            System.out.println(e);
        }
    }
}
```

Theory Assignment No: 04

- 1) What is Exception? Explain types of exceptions in Java.
- 2) Write difference between Error and Exception
- 3) How exceptions are handled in Java? Explain with example.
- 4) How exceptions are handled with multiple catch blocks.
- 5) Give the difference between throw and throws.
- 6) What is Thread? Explain Multithreading.
- 7) How we can create thread in Java?
- 8) What is synchronization? How thread is synchronized in Java?
- 9) What is File? How file manipulation is done in Java?
- 10) Write short note on:
 - 1) finally block
 - 2) throw keyword
 - 3) throws keyword
 - 4) Custom exception
 - 5) Thread life cycle
 - 6) Thread Priority
 - 7) Thread communication

Practical Assignment No: 07

- 1) Write a program that demonstrate use of checked exception.
- 2) Write a program that demonstrate use of unchecked exception.
- 3) Write a program that shows use of `ArrayIndexOutOfBoundsException`.
- 4) Write a program that shows use of `ArithmeticException`.
- 5) Write a program that shows use of `NullPointerException`.
- 6) Write a program that shows use of `NumberFormatException`.
- 7) Write a program that shows use of `FileNotFoundException`.
- 8) Write a program that demonstrate use of multiple catch blocks to handle different exceptions.
- 9) Write a program that demonstrate use of finally block.
- 10) Write a program that demonstrate use of throw keyword.
- 11) Write a program that demonstrate use of throws keyword.
- 12) Write a program that creates user defined exception. (Custom exception)
- 13) Write a Java program to create a method that takes a string as input and throws an exception if the string **does not contain vowels**.
- 14) Write a program that shows creating thread by extending Thread class.
- 15) Write a program that shows creating thread by implementing Runnable interface.
- 16) Write a program that shows use of `MAX_PRIORITY`, `MIN_PRIORITY` and `NORM_PRIORITY`.
- 17) Write a program that set user defined priority for thread and retrieve it.
- 18) Write a program that demonstrate multi-threading using synchronized block.
- 19) Write a program that demonstrate multi-threading using synchronized method.
- 20) Write a program that shows inter thread communication.
- 21) Write a Java program to create a producer-consumer scenario using the `wait()` and `notify()` methods for thread synchronization.
- 22) Write java program that Illustrate Priorities in Multithreading with the help of `getPriority()` and `setPriority()` methods.
- 23) Write a program to read the content of file.
- 24) Write a program to write the data into file.
- 25) Write a program to copy the content of one file into another.
- 26) Write a java program that demonstrate the use of `FileReader` and `FileWriter` class to read and write the data into file.
- 27) Write a java program illustrating the Byte Stream to copy contents of one file into another file.
- 28) Write a Java program that reads a file and **throws an exception if the file is empty**.

MCQ with Answers	
1. Which type of error is detected by the Java compiler? A) Runtime error B) Logical error C) Compile time error D) System error	16. Which exception is raised when accessing a null object? A) <code>IOException</code> B) <code>NullPointerException</code> C) <code>NumberFormatException</code> D) <code>InterruptedException</code>
2. Which of the following is a compile time error? A) Divide by zero B) Missing semicolon C) Array index out of bounds D) Null pointer access	17. Which exception is thrown for invalid array index? A) <code>ArithmeticException</code> B) <code>ArrayIndexOutOfBoundsException</code> C) <code>StringIndexOutOfBoundsException</code> D) <code>ClassCastException</code>
3. An exception is an event that occurs during: A) Compilation B) Design C) Execution	18. Multithreading means: A) Executing one task repeatedly B) Executing multiple programs

D) Debugging 4. Which exception occurs when a number is divided by zero? A) NullPointerException B) IOException C) ArithmeticException D) ClassNotFoundException 5. What happens if an exception is not handled? A) Program continues normally B) Compiler ignores it C) Program terminates abnormally D) JVM restarts 6. Checked exceptions are detected at: A) Runtime B) Compile time C) Design time D) Execution end 7. Which of the following is a checked exception? A) ArithmeticException B) NullPointerException C) IOException D) ArrayIndexOutOfBoundsException 8. Unchecked exceptions are detected by: A) Compiler B) JVM C) IDE D) Loader 9. Which package contains Exception class? A) java.io B) java.util C) java.lang D) java.net 10. Which keyword is used to handle exceptions? A) throws B) catch C) throw D) finally 11. The try block must be followed by: A) Only finally B) Only catch C) catch or finally D) return 12. Which block always executes? A) try B) catch C) throw D) finally 13. Which keyword is used to explicitly throw an exception? A) throws B) throw	C) Executing multiple threads simultaneously D) Executing one thread only 19. Which method starts a thread execution? A) run() B) execute() C) start() D) init() 20. Which method contains thread logic? A) start() B) run() C) sleep() D) yield() 21. Thread priority range is: A) 0–5 B) 1–10 C) 1–100 D) 0–10 22. Default thread priority is: A) 1 B) 10 C) 5 D) 0 23. Which keyword is used for thread synchronization? A) thread B) sync C) synchronized D) wait 24. Which method causes a thread to wait? A) notify() B) notifyAll() C) wait() D) sleep() 25. notifyAll() method wakes up: A) One thread B) Current thread C) All waiting threads D) JVM 26. Which package provides I/O classes? A) java.util B) java.io C) java.lang D) java.net 27. Byte streams work with: A) 16-bit data B) 32-bit data C) 8-bit data D) Unicode only 28. Character streams work with: A) Binary data B) 8-bit data C) 16-bit Unicode data
--	--

C) try D) catch 14. The throws keyword is used to: A) Handle exception B) Create exception C) Declare exception D) Ignore exception 15. Custom exception must extend which class? A) Throwable B) Error C) Exception D) RuntimeException	D) Images 29. Which class is used to read text files? A) FileInputStream B) FileReader C) InputStream D) Reader 30. Which class is used to write binary files? A) FileWriter B) FileReader C) FileOutputStream D) BufferedReader
---	---